



INT305 Machine Learning

Lecture 2

Linear Methods for Regression, Optimization

线性回归与优化方法

Sichen Liu

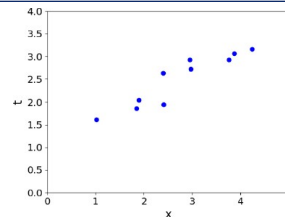
Department Intelligence Science

Sichen.Liu@xjtlu.edu.cn

Overview

- Second learning algorithm of the course: **linear regression**. 线性回归
 - Task: predict scalar-valued targets (e.g. stock prices) 预测标量值目标
 - Architecture: linear function of the inputs 架构: 输入量的线性函数
- While KNN was a complete algorithm, linear regression exemplifies a modular approach that will be used through this course:
 - choose a **model** describing the relationships between variables of interest 选择一个模型来描述感兴趣变量之间的关系
 - define a **loss function** quantifying how bad the fit to the data is 定义一个损失函数 用于量化数据拟合效果不佳的程度
 - choose a **regularizer** saying how much we prefer different candidate models (or explanations of data) 定义一个正则化器 以表明我们更偏好哪些候选模型
 - fit a model that minimizes the loss function and satisfies the constraint/penalty imposed by the regularizer, possibly using an **optimization algorithm** 拟合一个模型, 该模型最小化损失函数并满足正则化器施加的约束/惩罚条件 可能通过优化算法实现
- Mixing and matching these modular components give us a lot of new ML methods.

Supervised Learning Setup



在监督学习中

- In supervised learning: 输入 $x \in \mathcal{X}$, 特别是特征向量 协变量
- There is input $x \in \mathcal{X}$, typically a vector of features (or covariates)
 - There is target $t \in \mathcal{T}$, (also called response, outcome, output, class) 目标 $t \in \mathcal{T}$
 - Objective is to learn a function $f: \mathcal{X} \rightarrow \mathcal{T}$ such that $t \approx y = f(x)$ based on some data $\mathcal{D} = \{(x^{(i)}, t^{(i)}) \text{ for } i = 1, 2, \dots, N\}$

Linear Regression - Model

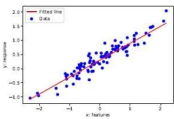
- **Model:** In linear regression, we use a linear function of the features $x = (x_1, \dots, x_D) \in \mathbb{R}^D$ to make predictions y of the target value $t \in \mathbb{R}$:

$$y = f(x) = \sum_j w_j x_j + b$$

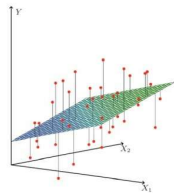
- y is the **prediction** 预期值
- w is the **weights** 权重
- b is the **bias** (or **intercept**) 偏移量

- w and b together are the **parameters**
- We hope that our prediction is close to the target: $y \approx t$.

What is Linear? 1 feature vs D features



- If we have only 1 feature:
 $y = wx + b$ where $w, x, b \in \mathbb{R}$.
- y is linear in x .



- If we have only D features:
 $y = w^T x + b$ where $w, x \in \mathbb{R}^D, b \in \mathbb{R}$
- y is linear in x .

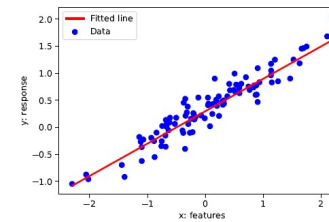
Relation between the prediction y and inputs x is linear in both cases.

无论是一维还是二维 线性回归的核心思想都是输入和输出之间保持线性关系

Linear Regression

We have a dataset $\mathcal{D} = \{(x^{(i)}, t^{(i)}) \text{ for } i = 1, 2, \dots, N\}$ where,

- $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_D^{(i)})^T \in \mathbb{R}^D$ are the inputs (e.g. age, height)
- $t^{(i)} \in \mathbb{R}$ is the target or response (e.g. income)
- Predict $t^{(i)}$ with a linear function of $x^{(i)}$:



- $t^{(i)} \approx y^{(i)} = w^T x^{(i)} + b$
- Different (w, b) define different lines.
- We want the "best" line (w, b) .
- How to quantify "best"?

w, b 的不同取值, 对应不同的直线或超平面
要找到最佳的直线, 即能最好地拟合数据的参数

$x^{(i)}$ 表示第 i 个样本
D 表示特征维的个数
如果有 N 个样本, 那么数据矩阵就是: $X \in \mathbb{R}^{N \times D}$
行 = 样本数 N
列 = 特征数 D

Linear Regression - Loss Function

- A **loss function** $\mathcal{L}(y, t)$ defines how bad it is if, for some example x , the algorithm predicts y , but the target is actually t .
 损失函数用来衡量预测值和真实值之间的差距
 y 是模型预测值, t 是实际值 损失函数越小, 代表预测越接近现实

- Squared error loss function**: 平方误差损失函数
 $\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$

- $y - t$ is the **residual**, and we want to make this small in magnitude.
 残差
- The $\frac{1}{2}$ factor is just to make the calculations convenient.

- Cost function**: loss function averaged over all training examples. 成本函数.

$$J(w, b) = \frac{1}{2N} \sum_{i=1}^N (y^{(i)} - t^{(i)})^2$$
 对于整个训练集 (N 个样本), 我们需要将所有样本的损失加起来取平均

$$= \frac{1}{2N} \sum_{i=1}^N (w^t x^{(i)} + b - t^{(i)})^2$$
 通过调整 w, b , 让 $J(w, b)$ 尽可能小, 得到最优参数 (w, b)

- Terminology varies. Some call "cost" *empirical* or *average loss*.

损失函数通常指单个样本的误差, 而成本函数通常指整个数据集的平均误差

Vectorization

- Notation-wise, $\frac{1}{2N} \sum_{i=1}^N (y^{(i)} - t^{(i)})^2$ gets messy if we expand $y^{(i)}$:
 如果直接将原始损失函数展开, 会出现很上下标, 难以阅读

$$\frac{1}{2N} \sum_{i=1}^N \left(\sum_{j=1}^D (w_j x_j^{(i)}) + b - t^{(i)} \right)^2$$

- The code equivalent is to compute the prediction using a for loop:
 使用代码也要 for 循环, 效率低下.

```
y = b
for j in range(M):
    y += w[j] * x[j]
```

- Excessive super/sub scripts are hard to work with, and Python loops are slow, so we **vectorize** algorithms by expressing them in terms of vectors and matrices.

$$w = (w_1, \dots, w_D)^t \quad x = (x_1, \dots, x_D)^t \quad \text{使用向量/矩阵来表达会更方便}$$

$$y = w^t x + b$$

- This is simpler and executes much faster:

```
y = np.dot(w, x) + b
```

python 代码.

Vectorization

Why vectorize? 原因

- The equations, and the code, will be simple and more readable. Gets rid of dummy variables/indices! 避免繁琐的下标, 更直观, 代码更短, 更可读
- Vectorized code is much faster 矢量化代码通常比python循环快几个数量级
 - Cut down on Python interpreter overhead 避免了值或器循环开销
 - Use highly optimized linear algebra libraries (hardware support) 使用优化好的线性代数库
 - Matrix multiplication very fast on GPU (Graphics Processing Unit) GPU上矩阵乘法可以大规模并行

Switching in and out of vectorized form is a skill you gain with practice.

- Some derivations are easier to do element-wise
- Some algorithms are easier to write/understand using for-loops and vectorize later for performance

Vectorization

- We can organize all the training examples into a **design matrix** X with one row per training example, and all the targets into the **target vector** t . 把所有样本输入组织成一个矩阵X 每一行是一个样本 每一列是某个特征在所有样本上的取值

$$X = \begin{pmatrix} \mathbf{x}^{(1)\top} \\ \mathbf{x}^{(2)\top} \\ \mathbf{x}^{(3)\top} \end{pmatrix} = \begin{pmatrix} 8 & 0 & 3 & 0 \\ 6 & -1 & 5 & 3 \\ 2 & 5 & -2 & 8 \end{pmatrix}$$

one feature across all training examples (pointing to the columns)

one training example (vector) (pointing to the rows)

- Computing the predictions for the whole dataset:

$$X\mathbf{w} + b\mathbf{1} = \begin{pmatrix} \mathbf{w}^T \mathbf{x}^{(1)} + b \\ \vdots \\ \mathbf{w}^T \mathbf{x}^{(N)} + b \end{pmatrix} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix} = \mathbf{y}$$

Vectorization

- Computing the squared error cost across the whole dataset:

$$y = Xw + b1$$

$$J = \frac{1}{2N} \|y - t\|^2 \quad \text{成本函数}$$

- Sometimes we may use $J = \frac{1}{2} \|y - t\|^2$ without a normalizer. This would correspond to the sum of losses, and not the averaged loss. The minimizer does not depend on N (but optimization might!). 这种情况相当于所有样本的损失求和, 而不是取平均值。最优解
- We can also add a column of 1's to design matrix, combine the bias and the weights, (w, b) 不变, 但在此时学习率可能需要调整, 损失函数不同 and conveniently write

$$X = \begin{bmatrix} 1 & [x^{(1)}]^T \\ 1 & [x^{(2)}]^T \\ \vdots & \vdots \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \text{and} \quad w = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

Then, our predictions reduce to $y = Xw$.

可以改写为 $X' = \begin{bmatrix} 1 & x_{11} & x_{21} \\ 1 & x_{12} & x_{22} \\ 1 & x_{13} & x_{23} \end{bmatrix}$, $w' = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$

假设 样本数 $N=3$
特征数 $D=2$
矩阵 $X = \begin{bmatrix} x_{11} & x_{21} \\ x_{12} & x_{22} \\ x_{13} & x_{23} \end{bmatrix}$, $w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$
 $y = \begin{bmatrix} x_{11}w_1 + x_{21}w_2 \\ x_{12}w_1 + x_{22}w_2 \\ x_{13}w_1 + x_{23}w_2 \end{bmatrix} = Xw + b1$

解决最小化问题 Solving the Minimization Problem

We define a cost function. This is what we'd like to minimize.

Two commonly applied mathematical approaches:

- Algebraic, e.g., using inequalities: 代数方法, 用不等式证明某点是最小值
 - To show z^* minimizes $f(z)$, show that $\forall z, f(z) \geq f(z^*)$ 为了证明 z^* 最小化了 $f(z)$, 要证明 $f(z^*) \leq f(z)$.
 - To show that $a = b$, show that $a \geq b$ and $b \geq a$ 如果需要证明 $a=b$, 可以证明 $a \geq b$ 且 $b \geq a$.
- Calculus: minimum of a smooth function (if it exists) occurs at a **critical point**, i.e. point where the derivative is zero. 在微积分中, 极值点出现在导数为0的地方。一维情况, 最小值出现在 critical point. $\frac{df}{dz} = 0$
 - Multivariate generalization: set the partial derivatives to zero (or equivalently the gradient). $\nabla f(w) = 0$

Solutions may be **direct** or **iterative**

- Sometimes we can directly find provably optimal parameters (e.g., set the gradient to zero and solve in closed form). We call this a **direct solution**.
- We may also use optimization techniques that iteratively get us closer to the solution. We will get back to this soon.

Direct: 直接解
通过数学公式直接求解最优解

例如在线性回归中, 可以解正规方程

优点: 一次性求出闭式解

缺点: 当特征维度很高时, 计算代价大 (矩阵求逆开销很大)

Iterative solution

使用优化算法 (比如梯度下降) 逐步接近

最优解

核心思想: 沿着梯度方向一步步更新参数

优点: 适合大规模数据和高维特征

缺点: 需要选择学习率, 迭代次数超参数

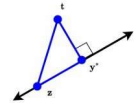
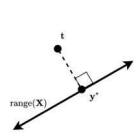
一个点到空间的最近点一定满足 $t - y^* \perp \text{range}(X)$

残差与子空间正交, 残差就是观测点到真实 $t \in \mathbb{R}^1$: 目标向量 (真实标签).

点的向量

Direct Solution I: Linear Algebra $X = \mathbb{R}^{N \times D}$ (行=样本, 列=特征), $w \in \mathbb{R}^D$ (权重向量).

- We seek w to minimize $\|Xw - t\|^2$, or equivalently $\|Xw - t\|$. Xw 的所有可能取值形成了 $\mathbb{R}^N$ 的一个子空间 $\text{range}(X)$. 我们希望找到离 t 最近的点.
- $\text{range}(X) = \{Xw | w \in \mathbb{R}^D\}$ is a D -dimensional subspace of $\mathbb{R}^N$.
- Recall that the closest point $y^* = Xw^*$ in subspace $\text{range}(X)$ of $\mathbb{R}^N$ to arbitrary point $t \in \mathbb{R}^N$ is found by orthogonal projection. 几何意义就是把点 t 投影到子空间 $y^* = Xw^*$.



$$(y^* - t) \perp Xw, \forall w \in \mathbb{R}^D$$

$$X^T(y^* - t) = 0$$

正交条件 $\text{range}(X)$ 上.

- We have $(y^* - t) \perp Xw, \forall w \in \mathbb{R}^D$. t 在空间中, y^* 是投影点.
- Why is y^* the closest point to t ?

- Consider any $z = Xw$. $y^* - t$ 与子空间中的所有向量正交.
- By Pythagorean theorem and the trivial inequality ($x^2 \geq 0$):

$$\|z - t\|^2 = \|y^* - t\|^2 + \|y^* - z\|^2$$

$$\geq \|y^* - t\|^2$$

$$\text{代入 } y^* = Xw^*$$

$$X^T(Xw^* - t) = 0$$

$$X^T X w^* = X^T t \quad (\text{normal equation})$$

Direct Solution I: Linear Algebra

- From the previous slide, we have $(y^* - t) \perp Xw, \forall w \in \mathbb{R}^D$.
- Equivalently, the columns of the design matrix X are all orthogonal to $(y^* - t)$, and we have that:

$$X^T(y^* - t) = 0$$

$$X^T X w^* - X^T t = 0$$

$$X^T X w^* = X^T t$$

$$w^* = (X^T X)^{-1} X^T t$$

如果 $X^T X$ 可逆, 我们就能得到

$$w^* = (X^T X)^{-1} X^T t$$

这是线性回归的直接解, 也叫最小二乘解.

- While this solution is clean and the derivation easy to remember, like many algebraic solutions, it is somewhat ad hoc.
- On the hand, the tools of calculus are broadly applicable to differentiable loss functions... 但是当特征维度 D 很大时, 计算 $(X^T X)^{-1}$ 非常耗时.

Direct Solution II: Calculus

偏导数. **Partial derivative:** derivative of a multivariate function with respect to one of its arguments. 对多元函数
固定其他变量
只对其中的一个变量
求导.

$$\frac{\partial}{\partial x_1} f(x_1, x_2) = \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2) - f(x_1, x_2)}{h}$$

- To compute, take the single variable derivative, pretending the other arguments are constant.
- Example: partial derivatives of the prediction $y = w_j x_j + b$

$$\begin{aligned} \frac{\partial y}{\partial w_j} &= \frac{\partial}{\partial w_j} \left[\sum_{j'} w_{j'} x_{j'} + b \right] & \frac{\partial y}{\partial b} &= \frac{\partial}{\partial b} \left[\sum_{j'} w_{j'} x_{j'} + b \right] \\ &= x_j & &= 1 \end{aligned}$$

Direct Solution II: Calculus

- For loss derivatives, apply the **chain rule**:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{d\mathcal{L}}{dy} \frac{\partial y}{\partial w_j} & \frac{\partial \mathcal{L}}{\partial b} &= \frac{d\mathcal{L}}{dy} \frac{\partial y}{\partial b} \\ &= \frac{d}{dy} \left[\frac{1}{2} (y - t)^2 \right] 4x_j & &= (y - t) \cdot 1 \\ &= (y - t)x_j & & \end{aligned}$$

- For cost derivatives, use **linearity** and average over data points:

$$\frac{\partial \mathcal{J}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)} \quad \frac{\partial \mathcal{J}}{\partial b} = \frac{1}{N} \sum_{i=1}^N y^{(i)} - t^{(i)}$$

- Minimum must occur at a point where partial derivatives are zero. 在最低解处, 梯度必须为0.

$$\frac{\partial \mathcal{J}}{\partial w_j} = 0 \quad (\forall j), \quad \frac{\partial \mathcal{J}}{\partial b} = 0.$$

(If $\partial \mathcal{J} / \partial w_j \neq 0$, you could reduce the cost by changing w_j)

梯度的意义指向函数上升最快的方向
(在优化时,我们沿着负梯度方向走,才能下降)

Direct Solution II: Calculus

- The derivation on the previous slide gives a system of linear equations, which we can solve efficiently.
- As is often the case for models and code, however, the solution is easier to characterize if we vectorize our calculus.
- We call the vector of partial derivatives the **gradient**. 把所有偏导数放到一起:
- Thus, the "gradient of $f: \mathbb{R}^D \rightarrow \mathbb{R}$ ", denoted $\nabla f(w)$, is: $\nabla f(w) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \vdots \\ \frac{\partial f}{\partial w_D} \end{bmatrix}$

$$\left(\frac{\partial}{\partial w_1} f(w), \dots, \frac{\partial}{\partial w_D} f(w) \right)^T$$

- The gradient points in the direction of the greatest rate of increase.
- Analogue of second derivative (the "Hessian" matrix):

$\nabla^2 f(w) \in \mathbb{R}^{D \times D}$ is a matrix with $[\nabla^2 f(w)]_{ij} = \frac{\partial^2}{\partial w_i \partial w_j} f(w)$.

$$\nabla^2 f(w) = \begin{bmatrix} \frac{\partial^2 f}{\partial w_1^2} & \dots & \frac{\partial^2 f}{\partial w_1 \partial w_D} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial w_D \partial w_1} & \dots & \frac{\partial^2 f}{\partial w_D^2} \end{bmatrix}$$

Direct Solution II: Calculus

- We seek w to minimize $J(w) = \frac{1}{2} \|Xw - t\|^2$ 成本函数:
- Taking the gradient with respect to w (see course notes for additional details) we get:

梯度: $\nabla_w J(w) = X^T X w - X^T t = 0$ $\nabla_w J(w) = X^T (Xw - t)$

- We get the same optimal weights as before:

闭式解

$$w^* = (X^T X)^{-1} X^T t$$

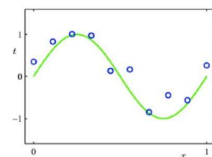
$$(X^T X)^{-1} (X^T X) = I$$

- Linear regression is one of only a handful of models in this course that permit direct solution.

在最优点处: $X^T X w = X^T t$

Polynomial Feature Mapping

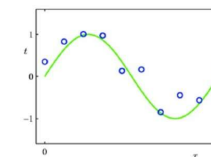
The relation between the input and output may not be linear.



- We can still use linear regression by mapping the input features to another space using **feature mapping** (or **basis expansion**). $\varphi(x): \mathbb{R}^D \rightarrow \mathbb{R}^d$ and treat the mapped feature (in $\mathbb{R}^d$) as the input of a linear regression procedure.
- Let us see how it works when $x \in \mathbb{R}$ and we use a polynomial feature mapping.
我们不改变线性回归的本质，而是通过特征映射 (Feature Mapping / Basis Expansion)，把原始输入变换到一个新的空间，在新空间里再做线性拟合。

Polynomial Feature Mapping

If the relationship does not look linear, we can fit a polynomial.

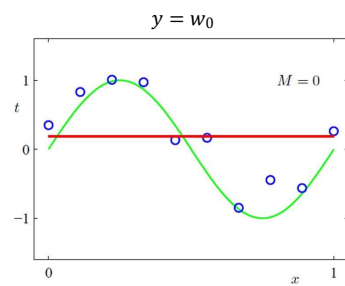


Find the data using a degree-M polynomial function of the form: 对于M阶多项式;

$$y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{i=0}^M w_i x^i$$

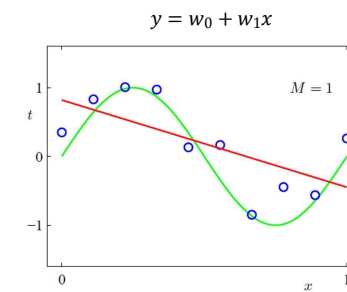
- Here the feature mapping is $\varphi(x) = [1, x, x^2, \dots, x^M]^T$. 对应的特征映射 $\varphi(x)$
- We can still use linear regression to find w , since $y = \varphi(x)^T w$ is linear with $w_0, w_1, \dots$
- In general, φ can be any function. Another example:
 $\varphi(x) = [1, \sin(2\pi x), \cos(2\pi x), \sin(4\pi x), \dots]^T$
 $\varphi(x)$ 可以是任意的非线性函数
虽然这个式子在原始的 x 空间里是非线性曲线，但在新特征空间 $\varphi(x)$ 里，它依然是一个线性模型，因为 $y = w^T \varphi(x)$

Polynomial Feature Mapping with $M = 0$



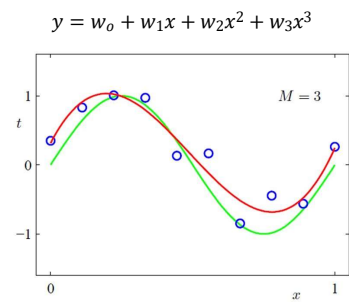
- Pattern Recognition and Machine Learning, Christopher Bishop.

Polynomial Feature Mapping with $M = 1$



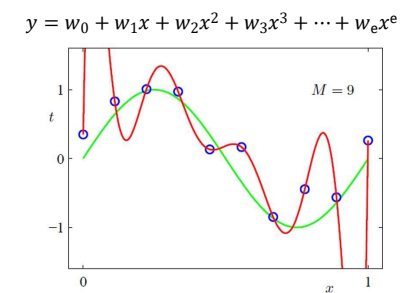
- Pattern Recognition and Machine Learning, Christopher Bishop.

Polynomial Feature Mapping with $M = 3$



- Pattern Recognition and Machine Learning, Christopher Bishop.

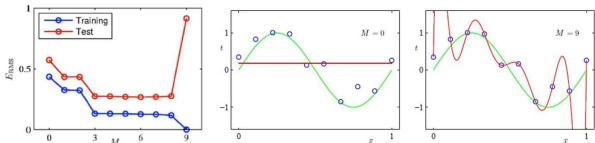
Polynomial Feature Mapping with $M = 9$



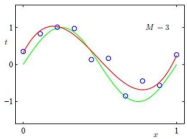
- Pattern Recognition and Machine Learning, Christopher Bishop.

Model Complexity and Generalization

Underfitting (M=0): model is too simple – does not fit the data. 欠拟合
Overfitting (M=9): model is too complex – fits perfectly. 过拟合

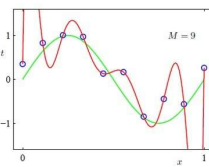


Good model (M=3): achieves small test error (generalizes well).



模型复杂度与泛化能力 Model Complexity and Generalization

	M = 0	M = 1	M = 3	M = 9
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43



绿色曲线: 阶数低的多项式 (M=3) 拟合平滑, 虽然不能完美通过所有点, 但整体趋势合理
红色曲线: 阶数高的多项式 (M=9) 精确通过每个数据点, 但在点与点之间剧烈震荡, 不符合数据真实性

- As M increases, the magnitude of coefficients gets larger.
- For M=9, the coefficients have become finely tuned to the data.
- Between data points, the function exhibits large oscillations.

Regularization

- The degree M of the polynomial controls the model's complexity.
- The value of M is a hyperparameter for polynomial expansion, just like k in KNN. We can tune it using a validation set.
- Restricting the number of parameters / basis functions (M) is a crude approach to control the model complexity.
- Another approach: keep the model large, but **regularize** it.
 - **Regularizer**: a function that quantifies how much we prefer one hypothesis vs. another.

模型复杂度由多项式决定: 多项式阶数越高, 模型越复杂, 越容易过拟合

从超参数, 通过验证集 (validation set) 来调整

通过正则化来约束模型参数, 从而保持模型复杂度的同时避免过拟合

L2 Regularization 正则化

- We can encourage the weights to be small by choosing as our regularizer the L^2 **penalty**.

$$\mathcal{R}(w) = \frac{1}{2} \|w\|_2^2 = \frac{1}{2} \sum_i w_i^2 \quad \text{惩罚项}$$

➤ Note: To be precise, the L^2 norm is Euclidean distance, so we're regularizing the squared L^2 norm.

- The regularized cost function makes a tradeoff between fit to the data and the norm of the weights.

$$J_{\text{reg}}(w) = J(w) + \lambda \mathcal{R}(w) = J(w) + \frac{\lambda}{2} \sum_i w_i^2$$

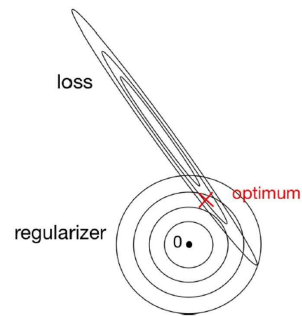
正则化后的目标函数 (成本函数) $\lambda > 0$ 控制正则化强度
 $J(w) = \frac{1}{2N} \|Xw - t\|^2$ λ 大 $\rightarrow$ 权重被压得更小, 代表模型越简单

- If you fit training data poorly, J is large. If your optimal weights have high values, $\mathcal{R}$ is large. 八小 $\rightarrow$ 更接近原始线性回归
- Large λ penalizes weight values more.
- Like M , λ is a hyperparameter we can tune with a validation set.

正则化的意思是: 在模型的目标函数中加入一个额外的约束项 (惩罚项), 用来限制模型参数的大小或复杂度, 从而防止模型过度拟合。

L2 Regularization

- The geometric picture:



相较于 L1 正则化直接取到变化过大的权重 w , L_2 会更加细致

L2 Regularized Least Squares: Ridge regression

For the least squares problem, we have $J(w) = \frac{1}{2N} \|Xw - t\|^2$. $\nabla_w J(w) = \frac{d}{dw} \cdot \frac{dw}{dJ} = X^T \cdot (Xw - t)$

- When $\lambda > 0$ (with regularization), regularized cost gives

$$w_{\lambda}^{\text{Ridge}} = \underset{w}{\operatorname{argmin}} \mathcal{J}_{\text{reg}}(w) = \underset{w}{\operatorname{argmin}} \frac{1}{2N} \|Xw - t\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

$$= (X^T X + \lambda I)^{-1} X^T t$$

- The case $\lambda = 0$ (no regularization) reduces to least squares solution!
- Note that it is also common to formulate this problem as $\underset{w}{\operatorname{argmin}} \frac{1}{2} \|Xw - t\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$ in which case the solution is $w_{\lambda}^{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T t$.

$w_{\lambda}^{\text{Ridge}}$ 与最小二乘解 $w^* = (X^T X)^{-1} X^T t$ 对比:

普通最小二乘: 可能在 $X^T X$ 不可逆时无解或不稳定

加入 λI : 保证了矩阵可逆 (即使 X 不满秩), 解更稳定

$$\begin{aligned} X^T X w - X^T t + \lambda w &= 0 \\ X^T X w + \lambda w &= X^T t \\ (X^T X + I \lambda) w^* &= X^T t \\ (X^T X + I \lambda)^{-1} (X^T X + I \lambda) w^* &= (X^T X + I \lambda)^{-1} X^T t \\ w^* &= (X^T X + I \lambda)^{-1} X^T t \end{aligned}$$

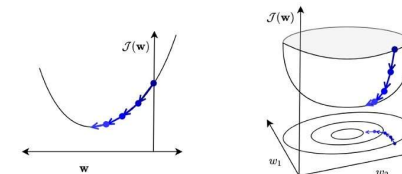
Conclusion so far

Linear regression exemplifies recurring themes of this course:

- Choose a **model** and a **loss function**
- Formulate an **optimization problem**
- Solve the minimization problem using one of two strategies
 - **Direct solution** (set derivatives to zero)
 - **Gradient descent** (next topic)
- **Vectorize** the algorithm, i.e. represent in terms of linear algebra
- Make a linear model more powerful **using features**
- Improve the generalization by adding a **regularizer**

Gradient Descent 梯度下降

- Now let's see a second way to minimize the cost function which is more broadly applicable: **gradient descent**. 目标是优化成本函数 $J(w)$, 但很多情况下没有像线性回归那样有解析解
- Many times, we do not have a direct solution: Taking derivatives of J w.r.t w and setting them to 0 doesn't have an explicit solution.
- Gradient descent is an **iterative algorithm**, which means we apply an update repeatedly until some criterion is met.
- We **initialize** the weights to something reasonable (e.g., all zeros) and repeatedly adjust them in the **direction of steepest descent**.



1. 初始化参数: 通常用零向量或小的随机值
2. 迭代更新: 按照某个规则调整参数直到收敛
3. 更新方向: 始终沿着梯度反方向更新 (最陡下降方向)

Gradient Descent

如果 $\frac{\partial J}{\partial w_i} > 0$, 说明增加 w_i 会让 J 增大 $\rightarrow$ 我们需要减小 w_i .
 如果 $\frac{\partial J}{\partial w_i} < 0$, 说明增加 w_i 会让 J 减小 $\rightarrow$ 我们需要增大 w_i .

- Observe:
 - If $\partial J / \partial w_j > 0$, then increasing w_j increases J .
 - If $\partial J / \partial w_j < 0$, then increasing w_j decreases J .
- The following update always decreases the cost function for small enough α (unless $\partial J / \partial w_j = 0$):

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

$\alpha > 0$: 学习率
 α 决定了每次更新的幅度
 太大可能震荡甚至发散, 太小收敛很慢.

- $\alpha > 0$ is a **learning rate** (or step size). The larger it is, the faster w changes.
- We'll see later how to tune the learning rate, but values are typically small, e.g., 0.01 or 0.0001.
- If cost is the sum of N individual losses rather than their average, smaller learning rate will be needed ($\alpha' = \alpha/N$).

Gradient Descent

- This gets its name from the **gradient**:

$$\nabla_w J = \frac{\partial J}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_D} \end{pmatrix}$$

$\nabla_w J$ This is the direction of fastest increase in J .

- Update rule in vector form:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}} \quad \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{t})$$

And for linear regression we have:

梯度下降更新公式

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{B}^{(i)}$$

- So gradient descent updates w in the direction of fastest **decrease**. 梯度下降, 沿负梯度方向
- Observe that once it converges, we get a critical point, i.e. $\frac{\partial J}{\partial \mathbf{w}} = 0$. 更新

为什么即使线性回归有闭式解, 仍然要用梯度下降

Gradient Descent for Linear Regression

- The squared error loss of linear regression is a convex function.
- Even for linear regression, where there is a direct solution, we sometimes need to use GD.
- Why gradient descent, if we can find the optimum directly?
 - GD can be applied to a much broader set of models
 - GD can be easier to implement than direct solutions
 - For regression in high-dimensional space, GD is more efficient than direct solution

- Linear regression solution: $(X^T X)^{-1} X^T y$
- Matrix inversion is an $O(D^3)$ algorithm
- Each GD update costs $O(ND)$
- Or less with stochastic GD (SGD, in a few slides)
- Huge difference if $D \gg 1$

闭式解只适用于线性回归, 而GD可以推广到非线性模型

代码上, GD不需要复杂的线性代数库来做矩阵求逆, 写几轮循环便可实现

主要是因为复杂度相关

最小二乘的闭式解是: $w^* = (X^T X)^{-1} X^T y$

这里涉及矩阵求逆, 时间复杂度是 $O(D^3)$, D 是特征维度

如果 D 很小, 闭式解非常快, 但如果 D

很大, 矩阵求逆会非常昂贵

梯度下降的更新公式 $w \leftarrow w - \alpha \lambda^{-1} X^T (Xw - y)$

每次更新的计算量是 $O(ND)$, 因为要算 Xw

如果迭代多次, 总复杂度是迭代次数 $\times O(ND)$

实际中, 样本数 N 比特征数 D 大很多, 代价

可以接受

梯度下降在 L_2 正则化下的更新公式 (权重衰减) Gradient Descent under the L_2 Regularization

- Gradient descent update to minimize J :

$$w \rightarrow w - \alpha \frac{\partial J}{\partial w} J \quad \text{在没有正则化情况下的梯度下降}$$

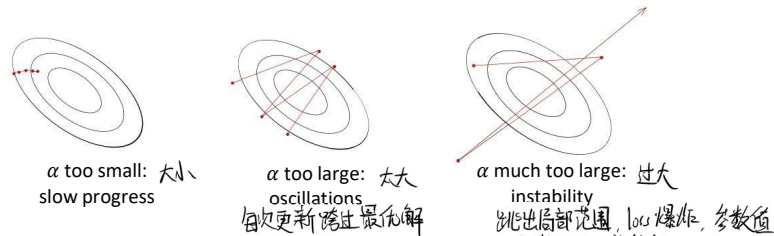
- The gradient descent update to minimize the L^2 regularized cost $J + \lambda R$ results in **weight decay**:

$$\begin{aligned} w &\rightarrow w - \alpha \frac{\partial}{\partial w} (J + \lambda R) && \text{weight decay} \\ &= w - \alpha \left(\frac{\partial J}{\partial w} + \lambda \frac{\partial R}{\partial w} \right) && \text{weight got smaller and smaller} \\ &= w - \alpha \left(\frac{\partial J}{\partial w} + \lambda w \right) && (1 - \alpha \lambda) w \text{ 让权重逐渐} \\ &= \underbrace{(1 - \alpha \lambda) w} - \alpha \frac{\partial J}{\partial w} && \text{衰减, 避免过大} \\ &&& - \alpha \frac{\partial J}{\partial w} \text{ 梯度下降的更新部分} \end{aligned}$$

$$\frac{\partial J}{\partial w} = X^T (Xw - y)$$

Learning Rate (Step Size) 学习率对梯度下降的影响

- In gradient descent, the learning rate α is a hyperparameter we need to tune. Here are some things that can go wrong:

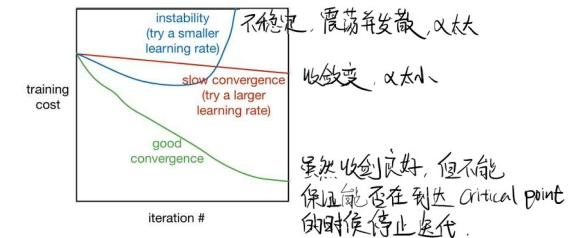


- Good values are typically between 0.001 and 0.1. You should do a grid search if you want good performance (i.e. try 0.1, 0.03, 0.01, ...).

实际中通常用自适应学习率优化器, 自动调节学习率.

Training Curves 训练曲线, 用来诊断优化问题

- To diagnose optimization problems, it's useful to look at **training curves**: plot the training cost as a function of iteration. 把训练过程中的损失随迭代次数变化画出来



- Warning: in general, it's very hard to tell from the training curves whether an optimizer has converged. They can reveal major problems, but they can't guarantee convergence.

随机梯度下降 (改进方法) Stochastic Gradient Descent

传统梯度下降 (batch gradient descent)

- So far, the cost function J has been the average loss over the training examples:

$$J(v) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}^{(i)} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y(x^{(i)}, v), t^{(i)})$$

$J(v)$ 定义为所有样本的平均损失

(v denotes the parameters; e.g., in linear regression, $w = (w, b)$)

- By linearity,
每次更新参数都要遍历所有 N 个样本 $\rightarrow$ batch training
- $$\frac{\partial J}{\partial w} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial w}$$
- N : 样本个数
 $\mathcal{L}^{(i)}$: 第 i 个样本的损失
 θ : 参数

- Computing the gradient requires summing over all of the training examples. This is known as **batch training**.
- Batch training is impractical if you have a large dataset $N \gg 1$ (e.g. millions of training examples.)! 如果数据集非常大 (几百万样本), 每次更新都要计算一遍, 非常慢

Stochastic Gradient Descent 为了提高效率, SGD 不再每次都遍历样本, 而是

- Stochastic gradient descent (SGD)**: update the parameters based on the gradient for a single training example,

1- Choose i uniformly at random,

$$2- w \rightarrow w - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial w}$$

- Cost of each SGD update is independent of N !
- SGD can make significant progress before even seeing all the data!
- Mathematical justification: if you sample a training example uniformly at random, the stochastic gradient is an **unbiased estimate** of the batch gradient:

$$\mathbb{E} \left[\frac{\partial \mathcal{L}^{(i)}}{\partial w} \right] = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial w} = \frac{\partial J}{\partial w}$$

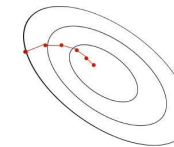
随机抽取一个样本
用它的梯度近似整个数据集的梯度
然后立即更新参数
优点: 每次更新的代价与 N 无关, 非常高效
缺点: 更新方向带有“噪音”, 并非最优方向

Stochastic Gradient Descent

- Problems with using single training example to estimate gradient:
 - Variance in the estimate may be high 使用单个样本噪声太大, 难以利用向量化运算
 - We can't exploit efficient vectorized operations
- Compromise approach: 折中方案 mini-batch 越大, 估计越接近真实梯度, 方差更小, 但计算开销更大.
 - Compute the gradients on a randomly chosen medium-sized set of training examples $\mathcal{M} \subset \{1, \dots, N\}$, called a **mini-batch**.
- Stochastic gradients computed on larger mini-batches have smaller variance.
- The mini-batch size $|\mathcal{M}|$ is a hyperparameter that needs to be set.
 - Too large: requires more compute: e.g., it takes more memory to store the activations, and longer to compute each gradient update
 - Too small: can't exploit vectorization, has high variance
 - A reasonable value might be $|\mathcal{M}| = 100$.

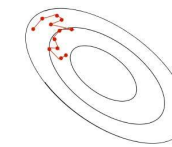
Stochastic Gradient Descent

- Batch gradient descent moves directly downhill (locally speaking).
- SGD takes steps in a noisy direction, but moves downhill on average.



batch gradient descent

每次更新方向稳定, 直到朝最优点下山
每次更新代价高, 尤其在大数据集上



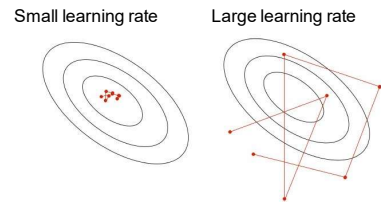
stochastic gradient descent

每次更新方向带有噪声, 轨迹“抖动”
但平均上仍向最优点靠近
计算快, 能在早期快速下降

因为SGD本身带噪声,因此学习率 α 不仅决定了收敛速度,还影响波动性 (fluctuation).

SGD Learning Rate

- In stochastic training, the learning rate also influences the **fluctuations**, due to the stochasticity of the gradients.



- Typical strategy:

- Use a large learning rate early in training so you can get close to the optimum
- Gradually decay the learning rate to reduce the fluctuations

前期用较大的学习率快速下降到接近最优解附近,节省大量训练时间

逐渐减小学习率,减少在最优解附近的波动.

构建机器学习模型的四个核心步骤

1. 选择模型 (Model): 选择输入和输出之间的关系, 比如线性模型 $y = w^T x + b$.

2. 定义损失函数 (Loss function): 衡量模型预测和真实数据的差距. 在线性回归中, 用平方误差 (squared error).

Conclusion 3. 选择正则化项 (regularizer): 防止模型过拟合, 提高泛化能力 (L^2 正则化).

- In this lecture, we looked at linear regression, which exemplifies a modular approach that will be used throughout this course: 4. 拟合/优化模型: 使用优化方法最小化损失函数, 比如梯度下降, 随机梯度下降.

➤ Choose a **model** describing the relationships between variables of interest (**linear**)

➤ Define a **loss function** quantifying how bad the fit to the data is (**squared error**)

➤ Choose a **regularizer** to control the model complexity/overfitting (L^2 , L^p regularization)

➤ Fit/Optimize the model (**gradient descent, stochastic gradient descent, convexity**)

机器学习方法可以看作模块化拼装: 模型结构 + 损失函数 + 正则化 + 优化方法.

- By mixing and matching these modular components, we can obtain new ML methods.
- Next lecture: apply this framework to classification.